

Project3 test

一、项目概述

本项目旨在实现一个快速且空间高效的单核内存分配器，其语义需与 `libc` 中的内存分配函数（`malloc`、`free`、`realloc`）保持一致。分配器通过 `trace` 文件重放真实程序的内存操作序列进行评分。

根据实验要求，性能指数 P 是空间利用率 (U) 和吞吐量 (T) 的加权几何平均值。官方计算公式为： $P = \exp(w \log U + (1 - w) \log(\min(1, T/T_{libc})))$ ，其中权重 w 设置为 0.5。在实际评分脚本中，这等价于： $\sqrt{\text{utilization} \times \text{throughput_ratio}}$ 。

- `utilization = max(40KB, max_total_size) / max(40KB, heap_size)`
- `throughput_ratio = student_throughput / (libc_throughput * 1.10)`

我们的核心限制条件包括：不得调用任何标准内存管理系统调用（如 `sbrk` 或 `libc` 的 `malloc`）；仅限单核运行；所有定义的全局和静态标量变量及复合数据结构的总大小不得超过 512 字节。

二、测试版目标与结果

2.1 目标

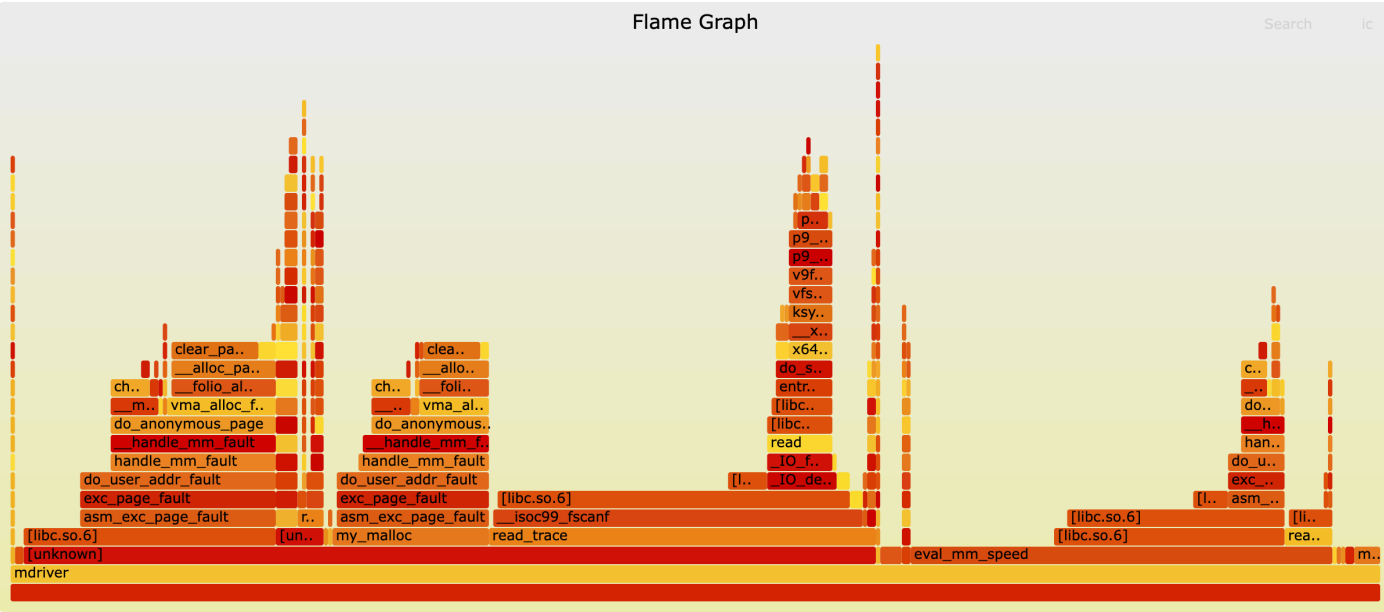
实现正确但不追求性能的分配器，通过所有 `trace` 的正确性验证。

2.2 耗时

设计	编码	调试	测试	会议
20min	1h	2h	10min	30min

2.3 热点图与性能瓶颈分析

通过 `perf record` 对测试驱动程序 `mdriver` 进行采样，并生成了如下的 CPU 周期热点数据：



1	+	30.79%	10.48%	mdriver	mdriver	[.] eval_mm_speed
2	+	30.16%	0.32%	mdriver	mdriver	[.] read_trace
3	+	30.16%	0.00%	mdriver	[unknown]	[k] 0xffffd4f9ffffd54c
4	+	30.16%	0.00%	mdriver	[kernel.kallsyms]	[.] asm_exc_page_fault
5	+	30.16%	0.00%	mdriver	[kernel.kallsyms]	[.] exc_page_fault
6	+	30.16%	5.08%	mdriver	[kernel.kallsyms]	[.] do_user_addr_fault
7	+	28.57%	0.00%	mdriver	[unknown]	[.] 0x6f20746f6e20646c
8	+	28.25%	0.32%	mdriver	libc.so.6	[.] __isoc99_fscanf
9	+	25.08%	0.32%	mdriver	[kernel.kallsyms]	[.] handle_mm_fault
10	+	24.76%	0.63%	mdriver	[kernel.kallsyms]	[.] __handle_mm_fault
11	+	23.81%	0.00%	mdriver	[kernel.kallsyms]	[.] do_anonymous_page
12	+	20.63%	0.00%	mdriver	libc.so.6	[.] 0x00007b00dff722c7
13	+	13.65%	0.32%	mdriver	[kernel.kallsyms]	[.] vma_alloc_folio
14	+	13.33%	0.00%	mdriver	libc.so.6	[.] 0x00007b00dfea7c2f
15	+	13.33%	2.22%	mdriver	mdriver	[.] my_malloc

根据上述采样数据与火焰图，测试版的性能瓶颈主要集中在以下三个方面：

- 严重的内核缺页异常开销（Page Fault）：高达 30.16% 的开销集中在 `asm_exc_page_fault` 及其底层的匿名页分配（`do_anonymous_page`，占 23.81%）。这表明分配器频繁地调用 `mem_sbrk` 扩展堆区，而在首次写入这些新分配的内存时，触发了大量的次要缺页中断（Minor Page Fault），导致大量 CPU 周期耗费在内核态的物理页分配上。
- Trace 文件解析与 I/O 瓶颈：`read_trace` 函数及其底层调用的 `__isoc99_fscanf` 占据了约 28.25% ~ 30.16% 的 CPU 周期。这部分属于测试驱动框架读取和解析 trace 文件的固有开销，但在分配器自身吞吐量较低时，这一比例显得尤为突出。
- 分配器核心函数的线性扫描损耗：`my_malloc` 占据了 13.33% 的开销。由于测试版采用了显式单空闲链表和首次适应（First Fit）策略，在面临大量内存碎片时，`my_malloc` 需要进行 $O(N)$ 的线性物理遍历来搜索可用块，导致了较高的指令执行开销和潜在的 Cache Miss。

2.4 结果

原始程序	测试版
Bump Allocator（无内存复用）	显式空闲双向链表 + 首次适应

2.5 评分

```
1 | GeometricMean(85.20 (util), 35.05 (tput)) = 54.65
```

三、 验证器（Validator）实现

为了支持黑盒测试，我们在 `validator.h` 中实现了一个外部验证器。验证器追踪每次 `malloc` / `realloc` 分配的 `payload` 范围，通过 `range_t` 链表维护所有活跃块的 `[lo, hi]` 区间。

3.1 `add_range()` – 合法性检查

对分配的 `payload` 逐一进行以下三项严格检查：

- 1. **对齐验证**：验证分配器返回的分配范围是否对齐到 8 字节。使用 `IS_ALIGNED(lo)` 宏（指针取模 `R_ALIGNMENT`），在 64 位下使用 `uint64_t` 避免截断。
- 2. **堆边界验证**：验证分配范围区间 `[lo, lo + size - 1]` 必须完全落在堆内（`[heap_lo(), heap_hi()]`）。
- 3. **重叠检测**：遍历 `ranges` 链表，判断新区间与已有区间是否有交集，确保不会把同一块内存分配给两个请求。

三项检查任一失败即调用 `malloc_error` 终止评测。通过后，以 $O(1)$ 时间复杂度将新区间插入链表头部。

3.2 `remove_range()` – 节点删除

释放时从 `ranges` 链表中移除对应的活跃块记录。采用二级指针 `range_t prevpp` 遍历链表，使得头节点与中间节点的删除逻辑完全统一，避免了额外的分支判断。

3.3 `eval_mm_valid()` – 数据完整性校验

针对 `realloc` 的特殊约束（必须将旧块中的数据完整保留到新块中），增加了数据填充与验证机制：

- **ALLOC**：用 `memset` 将 `payload` 填充为与块索引相关的字节模式。
- **REALLOC**：取出旧 `size` 与新 `size` 的较小值，逐字节比对新区块前部是否仍保留旧的填充模式。验证通过后重新填充新块。

四、 分配器（Allocator）核心设计

原始代码为无复用机制的 `bump allocator`。测试版将其重构为显式空闲双向链表 + 地址排序 + 首次适应（**First Fit**）的架构。为满足 512 字节的内存约束，全局变量仅维护一个 `free_list_head` 指针。

4.1 块结构设计

每个内存块（空闲或已分配）均包含 8 字节 header 和 8 字节 footer。由于要求 8 字节对齐，块大小的低 3 位恒为 0，因此 bit 0 被复用为分配标志位（1=已分配，0=空闲）。

1	[header:8B] [next:8B] [prev:8B] [...] [footer:8B]	← 空闲块（最小32B）
2	[header:8B] [payload:...]	[footer:8B] ← 已分配块

设计考量：

- **Footer 的必要性：**释放时需要利用前一块的 footer 来读取其状态和大小，从而实现 $O(1)$ 的向后合并。
- **最小块限制（32B）：** $header(8) + next(8) + prev(8) + footer(8) = 32B$ 。分割块时，若剩余空间不足 32B 则不进行分割，防止产生无法链入空闲表的小碎片。

4.2 空闲链表管理与合并（Coalesce）

空闲链表按地址升序排列，采用双向链接。

释放块时执行双向立即合并以最大化缓解外部碎片：

1. **向前合并：**检查前一块 footer 的分配位，若为空闲，将其从链表摘除，累加 size 并将当前指针回退。
2. **向后合并：**检查后一块 header 的分配位，若空闲则摘除并累加 size。
3. 更新合并后的 header/footer，并按地址顺序插入空闲链表。

4.3 核心接口实现

- `my_malloc`：计算真实需求大小（对齐并加上头部/尾部）。在空闲链表中进行首次适应搜索。若找到合适块，且剩余空间 $\geq 32B$ ，则进行分割（Split）。若无可用块，则调用 `mem_sbrk` 扩展堆空间。若请求大小为零，则安全返回空指针。
- `my_free`：定位至 header，清零 alloc 位，并调用 `coalesce` 进行合并与重新入链。释放空指针被定义为无操作。
- `my_realloc`：采用三级降级策略以尽量避免数据拷贝开销：
 1. **原地收缩：**若新尺寸更小且余量 $\geq 32B$ ，直接分割释放尾部。
 2. **向后合并扩展：**若下一物理相邻块为空闲，且合并后容量满足需求，直接合并（零拷贝）。
 3. **移动数据：**常规做法，`malloc` 新块 → `memcpy` 拷贝数据 → `free` 旧块。

五、堆检查器（Heap Checker）

指针处理不当易引发难以排查的内存损坏（memory corruptions）。我们实现了一个 `my_check` 函数来检验堆的一致性：

1. **物理遍历：**检查堆内每一个块的 header 与 footer 是否一致；验证是否存在未合并的连续空闲块；检查已分配的块之间是否存在重叠。
2. **链表遍历：**验证每个空闲列表中的块是否均被正确标记为 free；确保链表中的指针指向有效的堆地址。
3. **一致性核对：**比对物理遍历得到的空闲块总数与空闲链表节点数是否严格相等。